

L2 - 6.3. Monitoring AWS Logins

Task 1 - Introduction

Monitoring the control plane is the first step in securing your cloud environment. And within the control plane, authentication is the first area to focus on: who is logging in to AWS, from where, and which credentials they are using. This room walks you through the most common attacks against AWS identities and the defenses used to protect them.

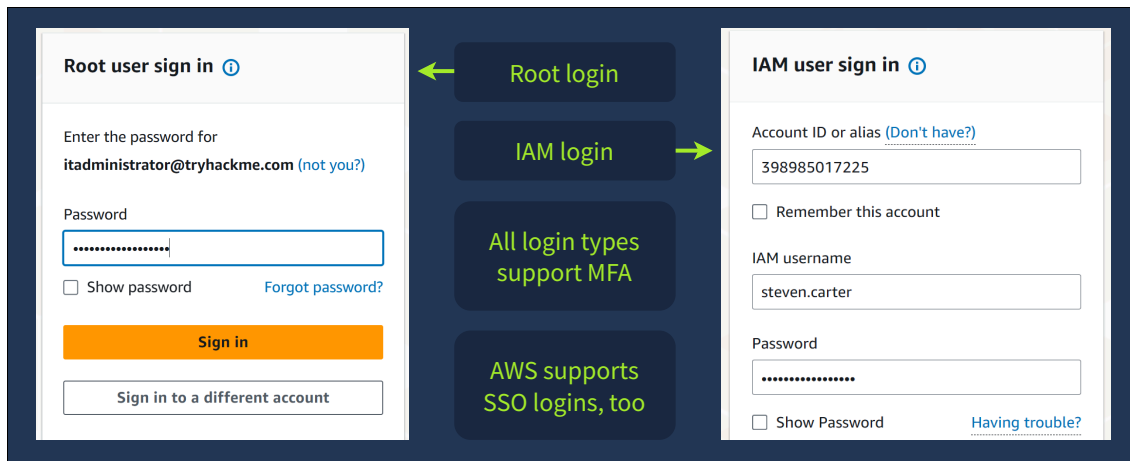
Task 2 - IAM and User Credentials

IAM and AWS Credentials

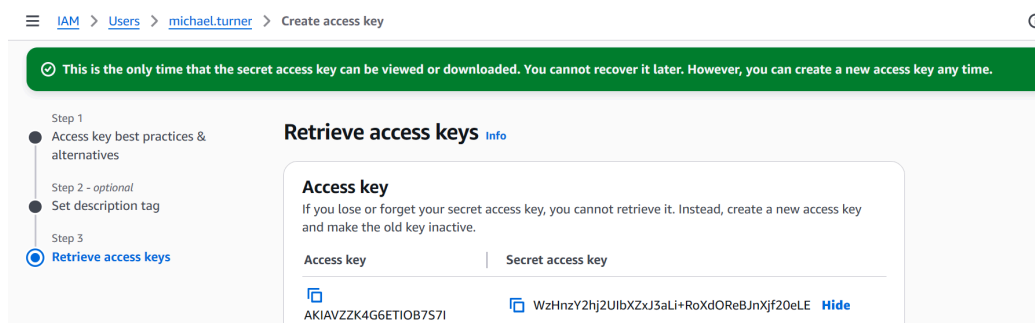
AWS Identity and Access Management (IAM) is a service, or rather a core AWS feature, that allows you to create cloud users, define policies and permissions, manage access to AWS resources, and much more. In this task, you will explore three methods to authenticate into AWS with IAM credentials, but first, remember its key components:

- **IAM User:** A person or application that needs AWS access. Just like the Active Directory user
- **IAM Group:** A collection of IAM Users, such as "EU DevOps Team" or "IT Support Contractors"
- **IAM Role:** An AWS identity that can be assumed by users or AWS services, more on it soon
- **IAM Policy:** A JSON document that defines permissions attached to users, groups, or roles

AWS Management Console: Secure web browser access to AWS resources requires signing in as an IAM user with appropriate policy-assigned permissions and MFA enabled—rather than using the root account—to reach the AWS Management Console.



IAM Access Keys: Programmatic AWS access via SDKs or the CLI relies on IAM access keys—composed of a public Access Key ID and a confidential Secret Access Key—which grant the executing application the exact same IAM permissions assigned to the associated user. Refer to the examples below.



Access Key Usage in AWS CLI

```
admin@it-srv:~$ aws configure                                     # Setting the access key
AWS Access Key ID [None]: AKIAVZZK4G6ETI0B7S7I
AWS Secret Access Key [None]: WzHnzY2hj2UI[...]admin@it-srv:~$
aws s3 ls                                                         # Listing all S3 buckets
2025-12-09 20:30:30 aws-cloudtrail-logs
2024-11-24 17:41:10 thm-db-backups
admin@it-srv:~$ aws s3 ls thm-db-backups --recursive             # Viewing the S3 bucket
2024-11-25 22:26:37 0 prod/
```

2025-12-17 01:07:14

790984

prod/backup.meta

2025-12-17 02:08:56

12089789743

prod/backup.tar.gz

IAM Roles and Policies: Imagine you have a few EC2 instances that resize uploaded images in the **user-avatars** S3 bucket. Would you create a new IAM service user, assign the correct permissions, generate a long-term access key, and hardcode it in EC2 in your application's source code? No, since it's tedious and insecure. Instead, you can create a **UserAvatarsProcessor** role, define its policy, and link it to the EC2 instances. No passwords or keys required! See an example in the image below:

The screenshot shows the AWS IAM console interface for creating a new role. It is divided into three main sections: 'Specify Trust Policy (Who can assume the role)', 'Specify Permissions Policy (What actions the role allows)', and 'Name and create the role (Now it can be assumed on demand)'. Below these are three panels: 'Step 1: Select trusted entities' showing a trust policy JSON snippet, 'Step 2: Add permissions' showing a permissions policy summary and JSON snippet, and 'Name, review, and create' showing the role name 'UserAvatarsProcessor' and a description 'A role for EC2 instances processing uploaded user avatars in S3'.

IAM roles eliminate long-term credentials by leveraging short-lived, automatically rotated STS tokens to deliver secure, dynamic, least-privilege access for AWS services, cross-account access, and SSO. Below are the key benefits of using IAM roles:

#1	STS tokens are temporary and valid for a few hours, unlike the permanent passwords and access keys
#2	Roles and STS tokens aren't hardcoded in files by design, reducing the risk of credential leaks (e.g. from GitHub (opens in new tab))
#3	Roles can be restricted to specific IP addresses, resources, or identities, which allows for granular access control

Other Sign-In Methods: Regardless of the third-party identity provider or SSO solution used, authentication ultimately standardizes on assuming an IAM role to

issue temporary STS credentials for resource access. There are also more advanced sign-in methods, such as via Google Workspace SSO, Active Directory, Amazon Cognito, and IAM Identity Center. Each introduces its own security risks and requires specific detection. However, at a high level, all of them rely on temporary STS credentials and the same role assumption mechanism you already read about.

Answer the questions below

| What type of credential is used to access AWS resources via CLI/SDK?

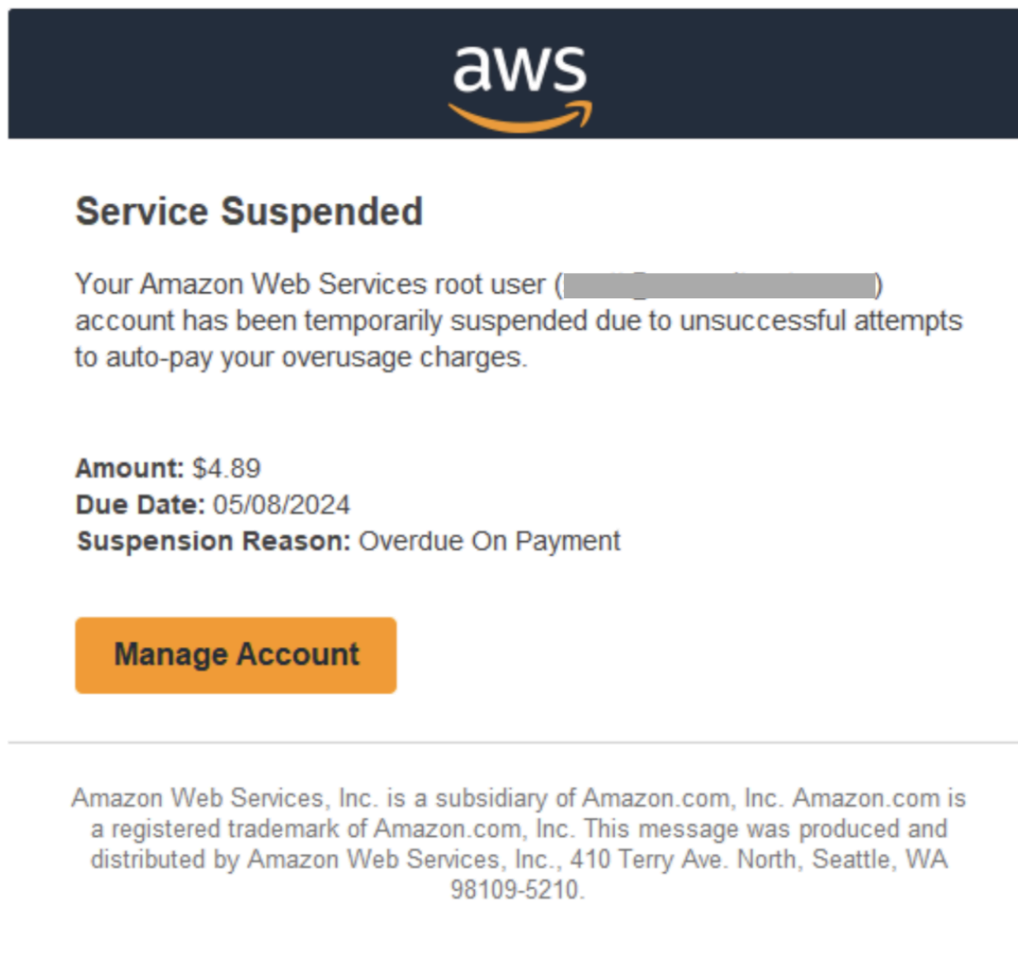
Access Key

| Which IAM identity type allows you to gain AWS permissions temporarily?

IAM Role

Task 3 - Monitoring Console Logins

Monitoring Console Logins: Monitoring AWS Management Console logins is essential because phishing campaigns targeting cloud credentials can bypass MFA through real-time proxies, giving attackers direct access to exfiltrate data, compromise resources, or deploy cloud ransomware. Refer to the [Wiz blog post\(opens in new tab\)](#) for a good example:



A phishing email leading to a fake AWS login page (Wiz)

Console Logins and CloudTrail: Management Console logins are the easiest to monitor, as CloudTrail logs every sign-in as the **ConsoleLogin** event. You can filter CloudTrail events using `eventName=ConsoleLogin` query and apply the regular investigation approach: verify whether the source IP is trusted or malicious, assess if the login time, user agent, and geolocation are typical for the user, and look for any unusual activity following the login. Below is an example of the ConsoleLogin event and a table that clarifies some of the fields:

```
// Most important fields of the ConsoleLogin event
{
  "eventTime": "2025-12-18T16:48:27Z",
  "eventName": "ConsoleLogin",
  "errorMessage": "Failed authentication",
```

```

"userIdentity": {
  "type": "IAMUser",
  "principalId": "AIDAVZZK4G6EYIICHBXXX",
  "arn": "arn:aws:iam::398985017225:user/john.doe",
  "accountId": "398985017225",
  "userName": "john.doe"
},
"sourceIPAddress": "12.42.87.160",
"userAgent": "Mozilla/5.0 [...]",
"requestParameters": null,
"additionalEventData": {
  "LoginTo": "https://console.aws.amazon.com/console/[...]",
  "MobileVersion": "No",
  "MFAUsed": "Yes"
},
"recipientAccountId": "398985017225"
}

```

Field Name	Explanation
errorMessage	If this field exists, it means the authentication was unsuccessful.
userIdentity.arn	A unique user identifier across all AWS accounts worldwide.
additionalEventData.MobileVersion	Whether the user logged in from the AWS mobile application.
additionalEventData.MFAUsed	Whether the user used MFA. It should always be set to "Yes"!

Key Monitoring Tips

- Monitor logins on the root user. Your IT team should use root for emergencies, not the daily routine
- Monitor console logins from service users. They are expected to use roles or access keys instead
- Monitor successful logins from VPN, Tor, cloud hostings, or otherwise suspicious IP addresses

- A focused, aggressive brute force might indicate that adversaries target your company

Practice: For this task, access the VM and search the **task3** Splunk index. You can start from the `index=task3 eventName=*` Splunk query. Search for **All Time** and note that field ordering may vary from the example.

Answer the questions below

| How many times did Thomas fail to log in to the AWS console?

11

`index=task3 eventName=*`

i	✓	Field	# of Va
>	☐	errorCode	
▼	☐	errorMessage	
Reports Top values Events with this field Top values by time Rare values			
		IAM user access not activated	24 26.667%
		Missing data for account: 398985017225	19 21.111%
		Failed authentication	11 12.222%
		User: arn:aws:iam::398985017225:user/otake.nao is not authorized to perform: health:DescribeEventAggregates on resource: * because no identity-based policy allows the health:DescribeEventAggregates action	6 6.667%

| Which other user logged in to the AWS console without MFA?

otake.nao

select field: `additionalEventData.MFAUsed thenselect user.name`

- `a` `tlsDetails.cipherSuite` 2
- `a` `tlsDetails.clientProvidedHostHeader` 23
- `a` `tlsDetails.tlsVersion` 2
- `a` `user` 8
- `a` `user_access_key` 93
- `a` `user_agent` 11
- `a` `user_arn` 7
- `#` `user_group_id` 1
- `a` `user_name` 6
- `a` `user_type` 5
- `a` `userAgent` 11
- `a` `userIdentity.accessKeyId` 93
- `#` `userIdentity.accountId` 1
- `a` `userIdentity.arn` 4
- `#` `userIdentity.principalId` 5
- `a` `userIdentity.sessionContext.attribute`
 - `s.creationDate` 7
- `a` `userIdentity.sessionContext.attribute`
 - `s.mfaAuthenticated` 2

user_name

6 Values, 96.975% of events Selected

Reports

[Top values](#)
[Top values by time](#)
[Rare values](#)

Events with this field

Values	Count	%
root	293	57.115%
otake.nao	201	39.181%
thomas.bennett	11	2.144%
AWSServiceRoleForResourceExplorer	4	0.78%
AWSServiceRoleForAmazonInspector2	3	0.585%
AWSServiceRoleForRDS	1	0.195%

5:00:01.000 PM additionalEventData: { [\[+\]](#)

Task 4 - Monitoring Access Keys

Risks of Access Keys: Long-lived access keys inadvertently committed to public repositories or hardcoded in app code are rapidly harvested by automated bots to compromise accounts, launch unauthorized compute workloads, and exfiltrate cloud data.

Cloud Email Abuse Campaign (Wiz)	In May 2025, threat actors abused AWS access keys, previously harvested by data stealers or discovered through public dorking, to start a phishing campaign. Using these keys, they configured Amazon Simple Email Service (SES) to mass-deliver phishing messages directly from the victims' AWS accounts. Save the blog post for future reading, it's very insightful!
Large Scale Cloud Extortion Operation (Unit 42)	In this global campaign, more than 70,000 AWS access keys were obtained from publicly accessible .env files. The keys were then used to exfiltrate and delete S3 objects from the target's S3 buckets. Finally, the threat actors uploaded a ransom note to the empty S3 buckets, urging the victims to pay a ransom to prevent the threat actors from selling the data.

Hello,

URGENT!

%100 of Your S3 files have been exfiltrated to our server.

We have all client personal information from json files ...

In order to prevent their SALE, you will need to make a payment in Bitcoin to us.

Note that if you do not make the payment, the files and personal information will be sold in the dark web and you will take all responsibility and consequences of it being leaked and reputation damage.

Once we receive the payment, we will delete all the files we have and you will never hear from us.

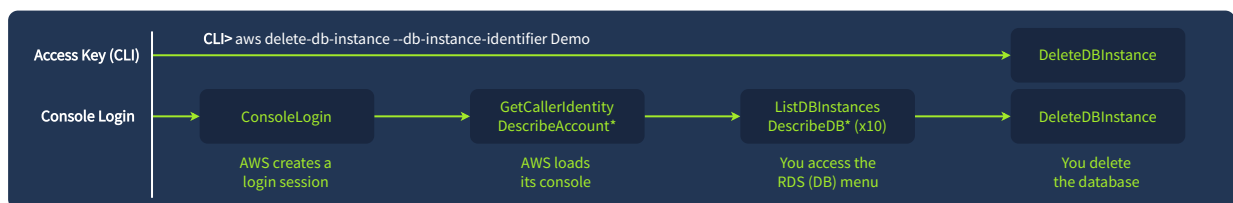
We are giving you the chance to resolve this quietly with no problems or headache.

To negotiate with us for a deal, contact us here:

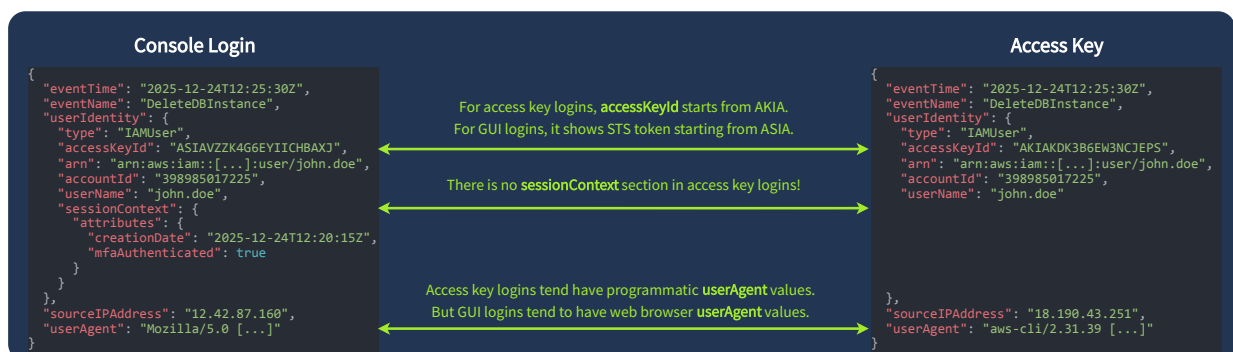
██████████

Ransom note left by the threat actor (Unit 42)

Monitoring Access Keys: Unlike browser access, programmatic activity using access keys bypasses the `ConsoleLogin` event entirely and generates laser-focused API calls—omitting the usual UI discovery traffic (`Get*`, `List*`, `Describe*`) in favor of direct, targeted actions.



Differentiate console logins from programmatic access key usage in CloudTrail by checking the `userAgent` string—browsers indicate console usage, while CLI/SDK tool identifiers reveal access key calls.



```
// Splunk query to filter only for access key logins
index=aws userIdentity.accessKeyId=AKIA*
```

```
// Alternative query to filter out console actions
index=aws NOT userIdentity.sessionContext.attributes.creationDate=*

```

SOC Pain Points: Analyzing automated CLI events from DevOps scripts and CI/CD pipelines creates major SOC friction because distinguishing routine deployment traffic from malicious automation requires deep context on script intent, expected schedules, and approved service behaviors. To help you identify whether an action is likely benign or malicious, consider the three practical tips below:

- **Correlate user purpose and action:** If the svc-s3-backup user suddenly starts launching EC2 instances instead of uploading objects to S3, this is a strong attack indicator. Especially if for the previous 90 days it never did such an action before. You can get more context about the user from the company documentation, [AWS tags\(opens in new tab\)](#), or its interactions with other AWS services.
- **Investigate the source IP address:** If the svc-gitlab-cicd user suddenly logs in from a DigitalOcean IP after years of operating from the GitLab IP range, this is a strong breach indicator. Service users don't change their location unless the service is migrated to another data center. You can get IP context on various TI services, such as [iplocation.net\(opens in new tab\)](#).
- **Investigate the AWS client (user-agent):** If your company heavily uses Java in AWS, you may see a lot of events with the aws-sdk-java/* user-agent. But if you see a sudden switch to, let's say Boto3/* (Python) or aws-cli/* (CLI) user-agent, it's either your team switched to another AWS client, or the user is compromised. If you can't recognize the user-agent, always google it first, and then confirm with your IT!

Practice: You will investigate a compromise of **michael.turner's** access key. The compromise led to an extortion attack on a private S3 bucket. Search for **All Time** and start from the `index=task4` query.

Answer the questions below

| What access key ID of Michael was used in the attack?

AKIAVZZK4G6EW3NCJENS

```
index=task4 NOT userIdentity.sessionContext.attributes.creationDate=*
```

select user.name filter userIdentity.accessKeyId

Select Fields

Select All Within Filter Deselect All Coverage: 1% or more ▾ X

i	✓ ▾	Field ▾	# of Values ▾
>	☐	requestParameters.key	27
▾	☐	userIdentity.accessKeyId	1
Reports			
Top values Events with this field			
Top values by time			
Rare values			
AKIAVZZK4G6EW3NCJENS 68 100%			
>	☐	user_access_key	1

What is the name of the S3 bucket accessed by the attackers?

ocr-passport-scan

run the query:

```
index=task4 NOT userIdentity.sessionContext.attributes.creationDate=* user_name="michael.turner"
```

select: eventSource="s3.amazonaws.com" filter 'bucket'

Select Fields

Select All Within Filter

Deselect All

Coverage: 1% or more ▾

bu X

i	✓ ▾	Field ▾	# of Values ▾
▼	☐	requestParameters.bucketName	1
Reports			
Top values			
Events with this field			
Top values by time			
Rare values			
ocr-passport-scan			
61			
100%			

How many files were exfiltrated and deleted by the adversary?

26

a object_category 1
a object_id 1
a product 1
a readOnly 1
recipientAccountId 1
a region 1
a requestID 1
a requestParameters.bucketName 1
a requestParameters.Host 1
a requestParameters.key 1
resources[].accountId 1
a resources[].ARN 2
a resources[].type 2
a responseElements.x-amz-server-side-encryption 1
a signature 1
a sourceIPAddress 1
a splunk_server 1
a src 1
a src_ip 1

requestParameters.key			X
1 Value, 100% of events		Selected	Yes No
Reports			
Top values			Top values by time
Events with this field			Rare values
Values			
	Count	%	
WHERE-ARE-MY-FILES.README	1	100%	

```

aOut 2/
a additionalEventData.CipherSuite 1
a additionalEventData.SignatureVersion 1
a additionalEventData.x-amz-id-2 62
a app 1
# aws_account_id 1
a awsRegion 2
a change_type 1
a command 5
# date_hour 1
# date_mday 1
# date_minute 6
a date_month 1
# date_second 15
a date_wday 1
# date_year 1
# date_zone 1
a dest 2
a dvc 1

```

command

5 Values, 100% of events Selected

Reports

Top values Top values by time Rare values

Events with this field

Values	Count	%
DeleteObject	26	41.935%
GetObject	26	41.935%
ListObjects	8	12.903%
ListBuckets	1	1.613%
PutObject	1	1.613%

host = [ce-splunk](#) | source = [cloudtrail.json](#) | sourcetype

Which file was uploaded to the bucket at the end of the attack?

WHERE-ARE-MY-FILES.README

```
index=task4 user_access_key=AKIAVZZK4G6EW3NCJENS eventName=PutObject
```

then filter requestParameters.key

```

a object_category 1
a object_id 1
a product 1
a readOnly 1
# recipientAccountId 1
a region 1
a requestID 1
a requestParameters.bucketName 1
a requestParameters.Host 1
a requestParameters.key 1
# resources[].accountId 1
a resources[].ARN 2
a resources[].type 2
a responseElements.x-amz-server-side-encryption 1
a signature 1
a sourceIPAddress 1
a splunk_server 1
a src 1
a src_ip 1

```

requestParameters.key ×

1 Value, 100% of events Selected

Reports

Top values Top values by time Rare values

Events with this field

Values	Count	%
WHERE-ARE-MY-FILES.README	1	100%

Which AWS service was used most by the user who **did not** use access keys?

Amazon Bedrock

```
index=task4 userIdentity.sessionContext.attributes.creationDate=*
```

```
# date_zone 1
a desc 1
a dest 13
a dvc 13
a errorCode 4
a errorMessage 10
a eventCategory 1
a eventID 99
a eventName 29
a eventSource 13
a eventTime 21
a eventType 1
a eventType 2
# eventVersion 2
a index 1
# linecount 1
a managementEvent 1
a msg 4
a object_category 2
a product 1
```

13 Values, 100% of events

Selected

Reports

Top valuesTop values by timeRare values

Events with this field

Top 10 Values	Count	%
bedrock.amazonaws.com	70	70.707%
health.amazonaws.com	6	6.061%
notifications.amazonaws.com	6	6.061%
ce.amazonaws.com	3	3.03%
freetier.amazonaws.com	3	3.03%
uxc.amazonaws.com	3	3.03%
ec2.amazonaws.com	2	2.02%
agreement-marketplace.amazonaws.com	1	1.01%
cost-optimization-hub.amazonaws.com	1	1.01%

Task 5 - Detecting IAM Role Abuse

IAM Roles for Services: The primary use of IAM roles is to allow AWS services to interact with each other. Let's continue with the example from task 2, where a Python script on your EC2 instance has to process user avatars in one of the S3 buckets. To make it happen, you link the **UserAvatarsProcessor** role to the instance. Then, the instance will automatically assume the role and use it whenever needed. In CloudTrail, EC2 actions would look like this:

```
// Simplified GetObject request made by the UserAvatarsProcessor role
{
  "eventTime": "2025-12-29T20:28:48Z",
  "eventSource": "s3.amazonaws.com",
  "eventName": "GetObject",
  "sourceIPAddress": "98.93.221.113",
}
```

```

Public IP of the EC2 instance
    "userAgent": "Boto3/1.34.46 md/Botocore#1.34.46[...] ",    //
Boto3 is AWS client for Python
    "userIdentity": {
        "type": "AssumedRole",                                //
Is always set to "AssumedRole"
        "principalId": "AR0AVZZK4G6ERIIVB7K4M:i-123456789",
        "arn": "arn:aws:sts::398985017225:assumed-role/UserAvatars
Processor/i-123456789",
        "accountId": "398985017225",
        "accessKeyId": "ASIAVZZK4G6E7I2DBMBB",                //
Always starts with "ASIA"
        "sessionContext": { [...] }
    },
}

```

Take a look at the last part of the ARN field, **i-123456789**. This is a role session name, and its meaning can vary depending on who assumes the role. By default, AWS sets it to the instance ID for EC2 and a user name for IAM users. However, it is possible to customize the session name and name it, for example, **MyCustomSession**. In this scenario, you won't know who assumed the role and will need to perform an additional investigation. Below are some ARN examples when using roles:

```

arn:aws:sts::[account-id]:assumed-role/[role-name]/[session-na
me]
arn:aws:sts::398985017225:assumed-role/UserAvatarsProcessor/i-
123456789    // Likely an EC2 instance ID
arn:aws:sts::398985017225:assumed-role/US-ITSupportRole/john.d
oe           // Likely an IAM user name
arn:aws:sts::398985017225:assumed-role/SuperAdmin/MyCustomSess
ion          // Need to investigate

```

AssumeRole Event: To reiterate, IAM roles can't operate on their own. During alert triage, you must always point to a real patient zero, a user or service that assumed the role: your IT admin, EC2 instance, Lambda function, or even AWS cloud itself. In

many cases, you can identify the actor by reading the last part of the ARN (the session name). However, if a custom session name is used, that approach won't work. Luckily, CloudTrail records the **AssumeRole** event, as shown in the example below:

```
// SuperAdmin role assumed by john.doe
{
  "eventTime": "2025-12-29T22:08:01Z",
  "eventSource": "sts.amazonaws.com",
  "eventName": "AssumeRole",
  "sourceIPAddress": "77.23.56.75",
  "userAgent": "aws-cli/2.31.39 [...]",
  "requestParameters": {
    "roleArn": "arn:aws:iam::398985017225:role/SuperAdmin",
    "roleSessionName": "MyCustomSession"
  },
  "userIdentity": {
    "type": "IAMUser",
    "principalId": "AIDAVZZK4G6E5AAVXXXXX",
    "arn": "arn:aws:iam::398985017225:user/john.doe",
    "accountId": "398985017225",
    "accessKeyId": "AKIAVZZK4G6EXJ0XXXXX",
    "userName": "john.doe"
  }
}
```

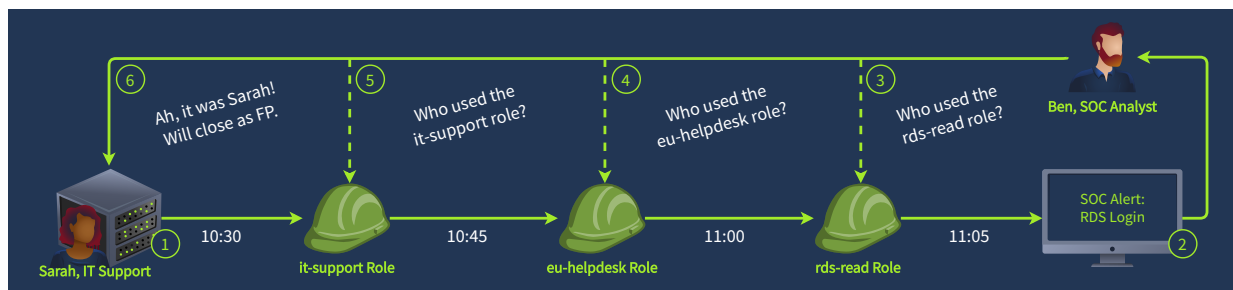
If the user above assumes the role and names the session **MyCustomSession**, all subsequent actions will be logged as coming from the role itself and won't refer to the original user name. As a result, you must trace back to the preceding **AssumeRole** event whenever you want to know who actually assumed the role. You will observe this behavior in the Splunk practice logs, and you are also encouraged to test it yourself using the AWS CLI.

IAM Role Abuse: IAM roles can be abused by adversaries. For example, imagine your EC2 instance gets compromised (e.g., via SSH brute-force attack) and an attacker gains full control over it. If that instance has an IAM role attached with S3

permissions, the attacker effectively inherits the same permissions as the EC2 and gains access to the S3 bucket.

- Detect a sudden change in the user-agent, same as with the access keys
- Hunt for first-seen API calls, especially those related to data exfiltration
- Protect the workloads, such as EC2 instances, that use IAM roles

Cross-Account Access: IAM roles are also used for cross-account access, which is fun to investigate. Imagine a scenario: Sarah from IT logged in and assumed the **it-support** role in the US account, then switched to the **eu-helpdesk** role in the EU account, and then assumed the **rds-read** role to debug a database issue. This is called role chaining, and it is common in large companies with complex environments. To "unchain" the events, you will have to analyze the **AssumeRole** events one by one until you find the real user or service behind the activity:



Practice: In this task, you will see how regular users and AWS services use IAM roles. Search for **All Time**, use the **task5** index, and start from the `index=task5` query.

Answer the questions below

Which EC2 instance ID used the **UserAvatarsProcessor** role?

i-0d2b8acdedc371589

```
index=task5 UserAvatarsProcessor then filter: userIdentity.arn
```

i	✓	Field	# of Values	Event Coverage	Type
>	☐	resources[].ARN	3	98.31%	String
>	☐	resources[].ARNPrefix	1	94.92%	String
✓	☐	userIdentity.arn	1	100%	String

Reports

[Top values](#)
[Events with this field](#)

[Top values by time](#)
[Rare values](#)

[arn:aws:sts::398985017225:assumed-role/UserAvatarsProc](#) 59 100%
 [essor/I-0d2b8acdedc371589](#)

>	☐	userIdentity.sessionContext.sessionIssuer.arn	1	100%	String
>	☐	user_arn	1	100%	String

Someone assumed the **EU-RemoteSupport** IAM role.

How did they name the role session?

SecretSession

filter: index=task5 eventName=AssumeRole , thenfilter userIdentity.arn

Select All Within Filter	Deselect All	Coverage: 1% or more	role	X	+ Extract Ne
i	✓	Field	# of Values	Event Coverage	Type
>	☐	eventtype	2	100%	String
>	☐	requestParameters.roleArn	1	100%	String
✓	☐	requestParameters.roleSessionName	1	100%	String

Reports

[Top values](#)
[Events with this field](#)

[Top values by time](#)
[Rare values](#)

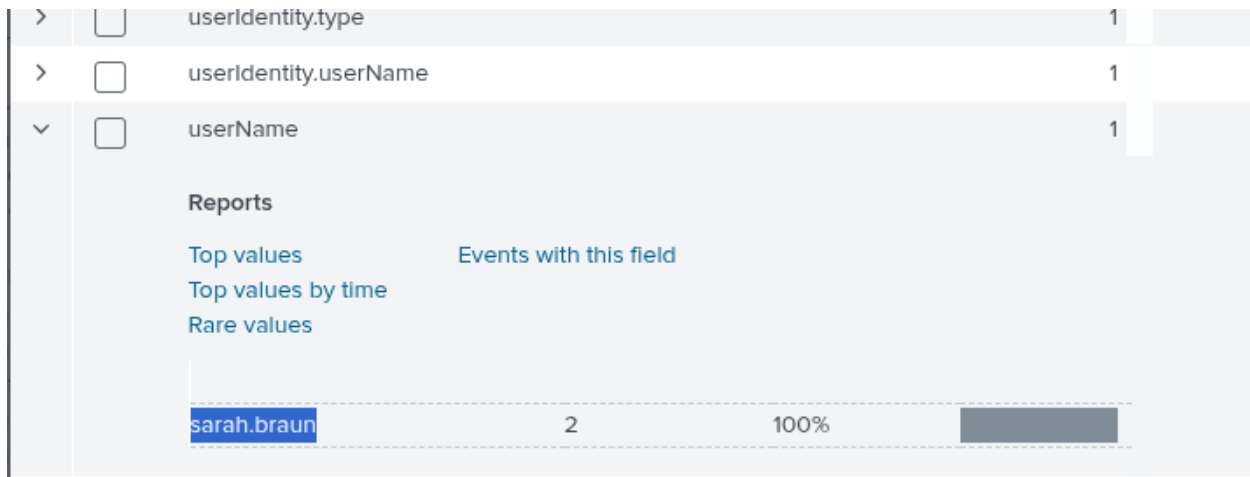
[SecretSession](#) 2 100%

>	☐	responseElements.assumedRoleUser.arn	1	100%	String
>	☐	responseElements.assumedRoleUser.assumedRoleId	1	100%	String
>	☐	user_role	1	100%	String

Which user assumed the IAM role from the question above?

sarah.braun

filter: index=task5 eventName=AssumeRole then filter userName



Task 6 - Detecting IAM Changes

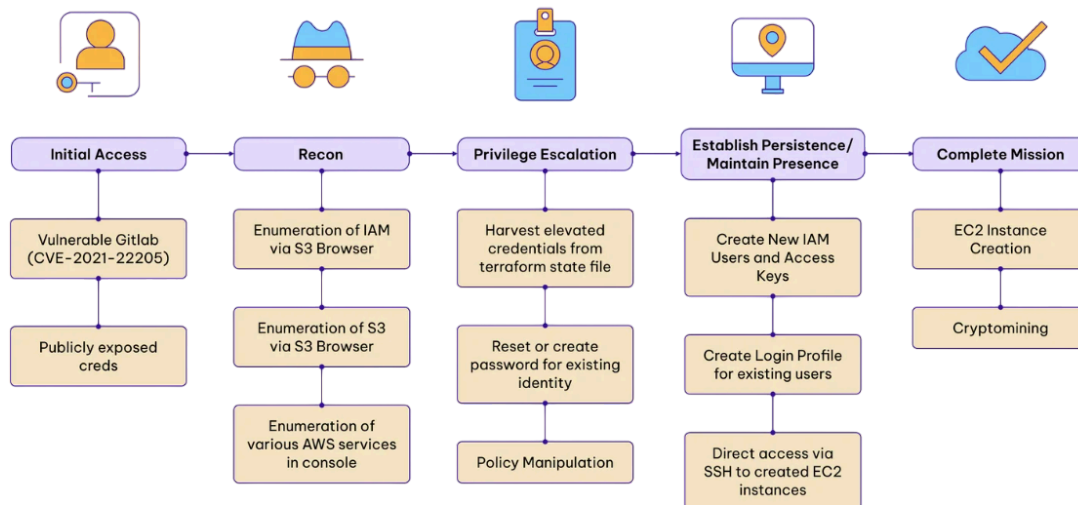
Early Threat Detection: The previous tasks focused on the moment of exploitation. However, the conditions for many attacks often form months or even years earlier. A common scenario: in 2023, an IT administrator created a root access key for testing and accidentally exposed it while recording a YouTube video guide. The key went unnoticed until 2025, when it was finally discovered and exploited. The ideal outcome for SOC would have been to detect and prevent the creation of the key in 2023, right?

IAM Misconfigurations: Proactively monitoring IAM modification events—like key creations, policy attachments, and trust updates—enables security teams to catch high-risk misconfigurations and privilege escalation paths long before they can be exploited. Below are the most common IAM events you should monitor for misconfigurations:

Event Names	Description	Suggestions
<code>PutUserPolicy</code> , <code>PutRolePolicy</code> , <code>PutGroupPolicy</code>	An existing AWS policy has been modified	Look for naming mismatches, such as "ReadOnlyAccess" policy modified to

Event Names	Description	Suggestions
		allow EC2 launch and termination
<code>Attach[User Group Role]Policy</code> , <code>AddUserToGroup</code>	An identity has been added to a group or assigned a new policy	Look for naming mismatches, such as "AdministratorAccess" policy attached to a newly hired intern
<code>DeleteVirtualMFADevice</code> , <code>DeactivateMFADevice</code>	A user deactivated their own, or someone else's MFA	Such events are legitimate only if followed by a new MFA registration (e.g. when a user loses their mobile phone and needs to set up a new MFA device)
<code>CreateAccessKey</code> , <code>UpdateAccessKey</code>	A user created or activated IAM access key	Always verify with your IT if the access key was expected, especially for privileged users. Leaked access keys are the most common Initial Access vector in AWS

Real-World Example: The [Permiso blog post](#)(opens in new tab) shows a great example of how IAM features can be abused both for Initial Access and Persistence. Here, the threat actors accessed the victim's AWS environment via a leaked access key and then created another access key for persistence, in case the first one is deactivated by the IT team. Check out the diagram below or the original blog post for more details:



Attack lifecycle of the GUI-vil threat group (Permiso)

Key Monitoring Points

- Monitor root user activity and prevent all attempts to create root access keys
- Monitor the learned IAM events, especially the **CreateAccessKey** one
- Always communicate with your IT about the potential misconfigurations

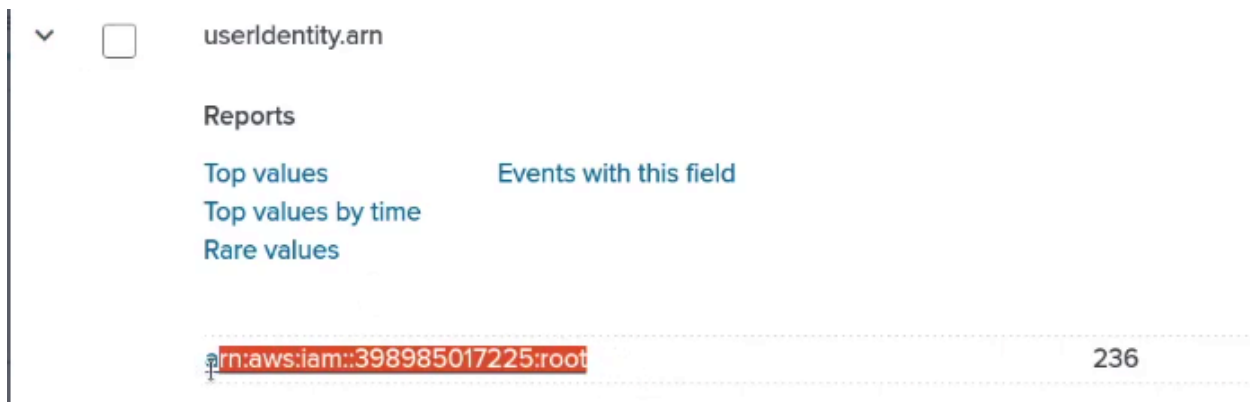
Practice: You will audit logs of an insecure integration between Splunk and CloudTrail. Search for **All Time**, use the **task6** index, and start from the `index=task6` query.

Answer the questions below

Under which ARN does the Splunk integration authenticate? SOC Note: This is an exceptionally insecure configuration!

arn:aws:iam::398985017225:root

run the query: index=task6 filter userIdentity.arn



When was the over-privileged integration access key created?

2025-12-29 19:59:23

index=task6 eventName=CreateAccessKey

Task 7 - Conclusion

By mastering how attackers exploit AWS console logins, access keys, and IAM roles using CloudTrail telemetry, you now hold the core blueprint for investigating initial access across any cloud platform. The same fundamental patterns apply directly to GCP Service Accounts and Azure Managed Identities, giving you a strong foundation to detect and defend against post-exploitation activities like extortion and cryptomining.